

## 27A – M7 Customers & Payment Recording: Implementation Guide

Everything the M7 developer needs: the customers schema as DDL, seed data, package layout, the API slice, handler specifications with pseudocode for the four that matter (reuse detection, the account-tender consumer, repayment allocation, anonymisation), the customer snapshot, the till contracts for account tender and repayment, screens, tests and tickets. Self-contained; restated material is marked.

| Field            | Value                                                                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Document         | 27A – implementation guide for 27 M7 Customers & Payment Recording v0.1                                                                             |
| Version          | v0.1 – for the M7 developer; section 7.3 is the contract with the till track                                                                        |
| Date             | 9 September 2026                                                                                                                                    |
| Prerequisite     | Sprint 0; M1 contract (responsible officer, locations); M6 receipt bundle shape (26A §3, tenders); kernel document base, A4 renderer, notifications |
| Estimated effort | About 17 developer-days for one full-stack developer                                                                                                |
| Restated content | Marked <i>restated from doc X §Y</i> .                                                                                                              |

**Read this first.** Two things must be right from the first commit: personal data never leaks into events, logs or snapshots beyond what the till needs to serve the customer (name, phone, balance, limit), and the posting ledger is insert-only with the balance as a recomputed cache. The privacy tests in Section 9 run in CI; a payload containing a phone number or a NIC fails the build.

## 1. What you are building

*Restated from doc 27 §1.* The registered customer, the credit account per MPCS, append-only postings from till tenders and repayments, customer payment receipts with oldest-first allocation, statements, the banking record, and the privacy workflows.

| You build                                                                                                                                                                           | You do not build                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Schema customers: customer, customer_phone, customer_consent, customer_account, account_posting, allocation, doc_customer_payment (extension), data_subject_request, banking_record | Membership of any kind; loans; bank integration; the till's tender screen (M6 renders your rules) |
| 18 commands, 8 queries, 8 event families; 2 consumers; 2 jobs; the customer snapshot contributor; the CPR bundle hook                                                               | Inter-entity payment receipts (M4)                                                                |
| Nine web screens; the till-side specifications for lookup, account tender and repayment                                                                                             | –                                                                                                 |

## 2. What you inherit

| From                    | You use                                                                                                   | How                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------|
| 19A §7 document base    | DocumentService.issue for CPR and adjustment documents; DocumentLinks (REVERSES)                          | Payments and reversals      |
| 19A §8 sync             | Bundle ingestion hook for CPR; SnapshotContributor; change log (urgent on suspend)                        | Section 7                   |
| 19A §6 i18n, A4Renderer | Statement rendering in the customer's language                                                            | Statements                  |
| 19A §10 notifications   | SMS summary via M9's rules on customer_statement.issued.v1                                                | Statements                  |
| 19A §3 permissions      | Sod.assertDistinct for adjustments; MFA flags                                                             | Adjustments, limits         |
| M1 query                | Responsible officer per entity; MPCS scope; locations of an MPCS (snapshot fan-out)                       | Privacy requests; snapshot  |
| M6 bundle               | receipt.issued/refunded.v1 with doc_receipt_tender rows (kind ACCOUNT, customer_account_id, offline flag) | Consumer (Section 6.2)      |
| Shared engine           | Money                                                                                                     | Never re-implement rounding |
| 17A §7 web shell        | DocumentHeader, ApprovalBar, ReasonCapture, StepUpDialog, Timeline                                        | Section 8                   |

### 3. Schema

*Restated from doc 27 §3 (CR-27-1). Location db/migration/m7customers/.*

```
-- V0001_customers.sql (RLS per 17A 6.3; note the identity-vs-account visibility split in 3.1)
CREATE TABLE customers.customer (
  customer_id uuid PRIMARY KEY, display_name text NOT NULL COLLATE en_icu, display_name_si text COLLATE
  si_icu, display_name_ta text COLLATE ta_icu,
  language text NOT NULL DEFAULT 'si', nic_hash char(64), nic_last4 char(4),
  attributes jsonb NOT NULL DEFAULT '{}' CHECK (pg_column_size(attributes) <= 2048),
  status text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','INACTIVE','ANONYMISED')),
  registered_by_entity_id uuid NOT NULL, registered_at timestamptz NOT NULL DEFAULT now(),
  owner_entity_id uuid NOT NULL; -- the registering MPCs; identity readable
  by any MPCs with an account (policy below)
CREATE TABLE customers.customer_phone (phone_id uuid PRIMARY KEY, customer_id uuid NOT NULL REFERENCES
  customers.customer,
  phone varchar(20) NOT NULL, is_primary boolean NOT NULL DEFAULT true, valid_from timestamptz NOT NULL
  DEFAULT now(), valid_to timestamptz,
  changed_by uuid, reason text, owner_entity_id uuid NOT NULL);
CREATE UNIQUE INDEX one_current_primary_phone ON customers.customer_phone (phone) WHERE valid_to IS NULL AND
  is_primary;
CREATE INDEX phone_history ON customers.customer_phone (phone, valid_to);
CREATE TABLE customers.customer_consent (consent_id uuid PRIMARY KEY, customer_id uuid NOT NULL REFERENCES
  customers.customer,
  purpose text NOT NULL CHECK (purpose IN ('CREDIT_ACCOUNT','STATEMENTS_NOTIFICATIONS')), granted_at
  timestamptz NOT NULL,
  granted_via text NOT NULL CHECK (granted_via IN ('TILL','WEB','PAPER')), withdrawn_at timestamptz, entity_id
  uuid NOT NULL, owner_entity_id uuid NOT NULL);
CREATE TABLE customers.customer_tag (customer_id uuid NOT NULL, tag_code varchar(40) NOT NULL, owner_entity_id
  uuid NOT NULL, PRIMARY KEY (customer_id, tag_code, owner_entity_id));

CREATE TABLE customers.customer_account (
  account_id uuid PRIMARY KEY, customer_id uuid NOT NULL REFERENCES customers.customer, mpcs_entity_id uuid
  NOT NULL,
  credit_limit numeric(14,2) NOT NULL DEFAULT 0, balance numeric(14,2) NOT NULL DEFAULT 0, -- cache,
  recomputed by handlers
  offline_cap numeric(14,2), terms_days smallint NOT NULL DEFAULT 30, hard_block boolean NOT NULL DEFAULT
  false,
  status text NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN','SUSPENDED','CLOSED')), opened_at timestamptz
  NOT NULL, opened_by uuid NOT NULL,
  owner_entity_id uuid GENERATED ALWAYS AS (mpcs_entity_id) STORED, UNIQUE (customer_id, mpcs_entity_id));

CREATE TABLE customers.account_posting (
  posting_id uuid NOT NULL, received_at timestamptz NOT NULL DEFAULT now(), account_id uuid NOT NULL,
  kind text NOT NULL CHECK (kind IN ('CHARGE','CREDIT','PAYMENT','ADJUSTMENT','REVERSAL')), amount
  numeric(14,2) NOT NULL CHECK (amount <= 0),
  business_date date NOT NULL, document_id uuid NOT NULL, document_line_id uuid, receipt_number text,
  location_id uuid, operator_user_id uuid,
  offline boolean NOT NULL DEFAULT false, limit_breached boolean NOT NULL DEFAULT false, allocated_amount
  numeric(14,2) NOT NULL DEFAULT 0,
  owner_entity_id uuid NOT NULL, PRIMARY KEY (received_at, posting_id)) PARTITION BY RANGE (received_at);
CREATE INDEX posting_by_account ON customers.account_posting (account_id, business_date);
GRANT INSERT ON customers.account_posting TO app_rw; GRANT UPDATE (allocated_amount) ON
  customers.account_posting TO app_rw;

CREATE TABLE customers.allocation (allocation_id uuid PRIMARY KEY, payment_posting_id uuid NOT NULL,
  charge_posting_id uuid NOT NULL,
  amount numeric(14,2) NOT NULL CHECK (amount > 0), created_at timestamptz NOT NULL DEFAULT now(), reversed_at
  timestamptz, owner_entity_id uuid NOT NULL);

CREATE TABLE customers.doc_customer_payment (document_id uuid NOT NULL, received_at timestamptz NOT NULL,
  account_id uuid NOT NULL,
  method text NOT NULL CHECK (method IN ('CASH','TRANSFER','DEPOSIT')), reference text, amount numeric(14,2)
  NOT NULL CHECK (amount > 0),
  allocation_mode text NOT NULL DEFAULT 'OLDEST_FIRST' CHECK (allocation_mode IN ('OLDEST_FIRST','SPECIFIC')),
  reversal_of uuid,
  owner_entity_id uuid NOT NULL, PRIMARY KEY (received_at, document_id)) PARTITION BY RANGE (received_at);

CREATE TABLE customers.data_subject_request (request_id uuid PRIMARY KEY, customer_id uuid NOT NULL, kind text
  NOT NULL CHECK (kind IN ('ACCESS','ERASURE')),
  received_at timestamptz NOT NULL, received_by uuid NOT NULL, status text NOT NULL DEFAULT 'RECEIVED' CHECK
  (status IN ('RECEIVED','FULFILLED','REFUSED')),
  fulfilled_at timestamptz, fulfilled_by uuid, export_object_key text, refusal_ground text, owner_entity_id
  uuid NOT NULL);

CREATE TABLE customers.banking_record (record_id uuid PRIMARY KEY, entity_id uuid NOT NULL, location_id uuid,
  session_ids uuid[] NOT NULL,
  amount numeric(14,2) NOT NULL CHECK (amount >= 0), expected_amount numeric(14,2) NOT NULL, kind text NOT
```

```

    NULL CHECK (kind IN ('HANDOVER_TO_OFFICE', 'BANK_DEPOSIT')),
    deposited_by uuid NOT NULL, received_by uuid, bank text, reference text, deposit_date date NOT NULL,
    shortfall_reason text,
    status text NOT NULL DEFAULT 'RECORDED' CHECK (status IN ('RECORDED', 'RECONCILED')), reconciled_ref text,
    owner_entity_id uuid NOT NULL);
CREATE TABLE customers.banked_session (session_id uuid PRIMARY KEY, record_id uuid NOT NULL REFERENCES
customers.banking_record, owner_entity_id uuid NOT NULL);

-- visibility: customer identity rows readable by any entity holding an account for the customer (policy joins
customer_account);
-- account, postings, allocations, payments, DSAR, banking: OWN (the MPCs) plus FEDERATION_VIEW.

```

### 3.1 Seed data

```

# seed/m7/config.yaml customers.offline_account_cap 5000 | limit_policy WARN | default_terms_days 30 |
allocation_mode OLDEST_FIRST |
# phone_reuse_window_months 12 | statement_frequency MONTHLY | statement_channel
SMS_SUMMARY | retention_years 7 | banking_shortfall_review ANY
# seed/m7/permissions.yaml (appended): cus.customer.register (LOCATION, online) | cus.customer.manage (ENTITY)
| cus.account.manage (ENTITY, mfa)
# cus.account.adjust (ENTITY, mfa) | cus.account.adjust_approve (ENTITY, mfa) | cus.payment.record
(LOCATION, offline) | cus.payment.reverse (ENTITY, mfa)
# cus.statement.issue (ENTITY) | cus.privacy.record (ENTITY) | cus.privacy.fulfil (ENTITY, mfa) |
cus.banking.record (LOCATION) | cus.banking.reconcile (ENTITY)
# seed/m7/sod-pairs.yaml (appended to M1's): [cus.account.adjust, cus.account.adjust_approve, INSTANCE]
# seed/m7/notification-rules.yaml (handed to M9): customer_statement.issued -> SMS summary to the customer's
primary phone when STATEMENTS consent exists

```

## 4. Package layout

```

lk.coopfed.knoweb.m7customers/ @ApplicationModule(allowedDependencies =
{"kernel", "m1party::api", "m1party::query"})
api/ commands (18), events (8 families), value types (PhoneNumber - validated, never logged; NicHash)
query/ CustomerQueries (lookup, get, list, snapshot), AccountQueries (account, overdue, receivables,
ageing, statement data), BankingQueries
internal/
customer/ Customer, Phone, Consent, handlers, ReuseDetector, Anonymiser
account/ Account, handlers, LimitPolicy
ledger/ PostingService (insert-only, balance recache), Allocator (oldest-first, specific),
AgeingCalculator
payment/ CustomerPayment handlers, CprBundleHook (till-captured repayments)
consumers/ ReceiptTenderConsumer (receipt.issued/refunded -> CHARGE/CREDIT), SessionConsumer (expected
banking)
statement/ StatementJob, StatementBuilder
privacy/ DataSubjectRequest handlers, Exporter, RetentionGuard
banking/ BankingRecord handlers
snapshot/ CustomerSnapshotContributor
queries/ *QueriesImpl
web/ CustomerController, AccountController, PaymentController, StatementController,
PrivacyController, BankingController

```

## 5. API slice: openapi/m7customers.yaml (excerpt)

```

/v1/customers                                post registerCustomer {displayName, phone, language, consents[],
  via}  get lookup (phone | q, mpcsId)
/v1/customers/{id}                          get  patch updateAttributes/tags  post /phone {newPhone, reason,
  confirmedIdentity?} | /deactivate
/v1/customers/{id}/consents                 post recordConsent | delete withdrawConsent
/v1/customers/{id}/accounts                 post openAccount {mpcsId, creditLimit, termsDays, offlineCap, nic?}
/v1/accounts/{id}                           get (balance, limit, ageing, postings cursor)  post /limit |
  /hard-block | /offline-cap | /suspend | /reinstate | /close
/v1/accounts/{id}/adjustments               post postAdjustment {amount, reason}  /{adjId}/approve post
/v1/accounts/{id}/payments                  post recordCustomerPayment {method, amount, reference,
  allocationMode, specific?: [{chargePostingId, amount}]}
/v1/customer-payments/{docId}/reverse       post {reason}
/v1/accounts/{id}/statements                post issue {period, channel}  get list
/v1/mpcs/{id}/overdue                       get (bucket)  /receivables get  /ageing get
/v1/privacy/requests                        post record {customerId, kind}  /{id}/fulfil | /refuse post
/v1/banking/records                         post recordBanking {kind, sessionIds[], amount, bank?, reference?,
  depositDate, shortfallReason?}  /{id}/reconcile post
/v1/banking/unbanked-sessions               get (mpcsId)
# example - lookup response (till-safe shape; no NIC)
{ "customerId": "...", "displayName": "K. Perera", "language": "si", "phone": "07xxxxxxxx",
  "account": { "accountId": "...", "mpcsId": "...", "creditLimit": 15000.00, "balance": 6200.00, "available":
    8800.00, "offlineCap": 5000.00,
    "hardBlock": false, "status": "OPEN", "oldestUnpaid": "2026-08-20" }, "tags": ["regular"] }

```

## 6. Handler specifications

| Command                                                                 | Guards (in order)                                                                                                                                                 | Effect                                                          | Events                                         |
|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------|
| RegisterCustomer                                                        | online (interceptor: command not offline_allowed); phone valid E.164-normalised; ReuseDetector (6.1) passed or confirmed; CREDIT_ACCOUNT consent present          | insert customer, primary phone, consents                        | customer.registered.v1;<br>consent.recorded.v1 |
| ChangePhone                                                             | customer ACTIVE; new phone valid; reuse detection; reason                                                                                                         | close old phone row (valid_to), insert new primary              | customer.phone_changed.v1                      |
| DeactivateCustomer                                                      | no OPEN account with balance ≠ 0                                                                                                                                  | INACTIVE                                                        | customer.deactivated.v1                        |
| UpdateAttributes / TagCustomer                                          | owner MPCS; attributes size                                                                                                                                       | rows                                                            | customer.updated.v1                            |
| OpenAccount                                                             | MPCS scope; customer ACTIVE; consent; no existing account; NIC captured (hash + last4) when limit > 0; MFA                                                        | insert OPEN                                                     | account.opened.v1                              |
| AmendLimit / SetHardBlock / SetOfflineCap                               | reason; MFA when the limit increases                                                                                                                              | update; audit before/after                                      | account.limit_amended.v1                       |
| SuspendAccount / ReinstateAccount / CloseAccount                        | reason; close requires balance = 0 and no unallocated payments                                                                                                    | status; snapshot urgent on suspend                              | account.suspended/reinstated/closed.v1         |
| PostCharge / PostCredit (internal)                                      | from receipt tender lines (6.2)                                                                                                                                   | postings; balance recache; breach flag                          | account.charged/credited.v1                    |
| RecordCustomerPayment                                                   | account OPEN or SUSPENDED; amount > 0; specific allocations ≤ open charges; issuance (till-position series at a till via the bundle, ENTITY series in the office) | CPR document; PAYMENT posting; Allocator (6.3); balance recache | customer_payment.recorded.v1                   |
| ReverseCustomerPayment                                                  | RECORDED; MFA; reason                                                                                                                                             | REVERSES-linked CPR; REVERSAL posting; allocations reversed     | customer_payment.reversed.v1                   |
| PostAdjustment / ApproveAdjustment                                      | reason; SoD request ≠ approve; MFA                                                                                                                                | adjustment document; ADJUSTMENT posting on approval             | account.adjusted.v1                            |
| IssueCustomerStatement                                                  | period; channel per consent                                                                                                                                       | render; store key; notification via event                       | customer_statement.issued.v1                   |
| RecordDataSubjectRequest / FulfilAccess / FulfilErasure / RefuseRequest | officer scope; ACCESS: export; ERASURE: RetentionGuard (no open balance) then Anonymiser (6.4); MFA on fulfil; refusal ground                                     | request status; anonymised customer                             | dsar.*.v1; customer.anonymised.v1              |

| Command                        | Guards (in order)                                                                                     | Effect                                           | Events                          |
|--------------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------|---------------------------------|
| RecordBanking / MarkReconciled | sessions CLOSED and unbanked; expected from M6 session facts; shortfall reason when amount < expected | record; banked_session rows; REVIEW on shortfall | banking.recorded/rec onciled.v1 |

## 6.1 Reuse detection

```
ReuseDetector.check(phone, forCustomer?): Outcome
  current = phones.currentPrimaryHolder(phone) // valid_to IS NULL
  if (current != null && current.customerId != forCustomer) return CONFLICT(current) // must
    ChangePhone on the other customer first
  recent = phones.holdersWithin(phone, config.reuseWindowMonths) // valid_to within the window
  if (recent.isNotEmpty()) return NEEDS_CONFIRMATION(recent.map { it.customerId }) // operator
    confirms different person: name + NIC last4 when an account exists
  return CLEAR
// on confirmation the command carries confirmedIdentity = true and the audit event records the previous
  holders' ids (never their names)
```

## 6.2 Receipt tender consumer

```
@EventConsumer(types={"receipt.issued.v1","receipt.refunded.v1","receipt.voided.v1"}, consumer="m7.tenders")
for tender in bundle.tenders where kind == ACCOUNT:
  acct = accounts.lock(tender.customerAccountId) // FOR UPDATE
  kind = when (event) { issued -> CHARGE; refunded, voided -> CREDIT }
  amount = if (kind == CHARGE) tender.amount else -tender.amount
  breach = kind == CHARGE && (acct.balance + amount > acct.creditLimit)
  postings.insert(Posting(acct, kind, amount, businessDate = bundle.document.businessDate, document =
    bundle.document.id, receiptNumber, location, operator,
    offline = tender.offline, limitBreached = breach))
  acct.balance += amount; accounts.save(acct)
  if (breach) { audit REVIEW ACCOUNT_LIMIT_BREACH(offline = tender.offline); if (acct.hardBlock &&
    tender.offline) audit ALERT HARD_BLOCK_BYPASSED_OFFLINE }
  events.publish(kind == CHARGE ? AccountCharged : AccountCredited)
  changelog.record(locationsOf(acct.mpcs), "customer", acct.customerId, UPSERT) // balance to tills
  (coalesced)
```

## 6.3 Allocator and 6.4 Anonymiser

```
Allocator.allocate(paymentPosting, mode, specific?):
  open = postings.openCharges(account) ordered by business_date, received_at // amount - allocated_amount >
  0
  remaining = payment.amount
  targets = if (mode == SPECIFIC) specific.map { charge(it.id) to it.amount } else open.map { it to
    min(it.open, remaining) }
  for (charge, amt) in targets while remaining > 0: a = min(amt, charge.open, remaining)
    allocations.insert(payment, charge, a); charge.allocated += a; payment.allocated += a; remaining -= a
  // remaining > 0 stays unallocated on the payment (credit balance); ageing uses charge.open and
    business_date
Anonymiser.apply(customer):
  require(accounts.noneOpenWithBalance(customer), "erasure.open_balance")
  customer.displayName = "Customer"; displayName_si/ta = null; nic_hash = null; nic_last4 = null; attributes =
    {}; status = ANONYMISED
  phones.closeAll(customer, reason = "ERASURE"); consents.withdrawAll(customer); tags.clear(customer)
  postings, allocations, payment documents: untouched (retention); statements object keys: deleted; audit
    records the request id only
```

## 7. Queries, jobs, snapshot and the till contract

| Item                         | Implementation                                                                                                                                                                                 |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LookupCustomer               | By E.164 phone (current primary) or by name trigram within the MPC's customers; returns the till-safe shape of Section 5; audited when called from accounts screens (not from the sales path). |
| AgeingCalculator             | For each open charge: bucket by today - business_date (0-30, 31-60, 61-90, 90+) on the unallocated amount.                                                                                     |
| StatementJob (per frequency) | Accounts with movement in the period → StatementBuilder → A4Renderer in the customer's language → customer_statement.issued.v1 (M9 sends the SMS summary where consented).                     |
| SessionConsumer              | till_session.closed.v1 → expected banking per session cached for UnbankedSessions.                                                                                                             |
| BankingShortfallJob (daily)  | Sessions closed more than 3 days ago and unbanked → REVIEW to the MPC's accounts role.                                                                                                         |

### 7.3 Snapshot contributor and till contract

```
CustomerSnapshotContributor: tables(): ["customer"]
rowsFor(location): accounts of the owning MPC's with status OPEN or SUSPENDED (and the identity rows behind them) ->
{ customerId, displayName (shop language with fallback), phone (primary), language, accountId,
  creditLimit, balance, offlineCap, hardBlock, status, tags[] }
// never: nic_hash, nic_last4, attributes, consent details
change-log: customer.*, account.*, account.charged/credited (coalesced 5 min), customer_payment.* -> UPSERT
for the MPC's locations; account.suspended -> urgent
Till contract (M6): ACCOUNT tender allowed when status OPEN, amount <= offlineCap (offline) and amount <=
(creditLimit - provisionalBalance) unless !hardBlock (warn);
provisionalBalance = snapshot.balance + local ACCOUNT tenders since snapshot - local CPR amounts since
snapshot
CPR bundle (till -> central): document(CPR, till-position series) + doc_customer_payment{accountId, method
CASH, amount, reference?, allocationMode OLDEST_FIRST}
hook: CprBundleHook -> PAYMENT posting + Allocator; a CPR against a CLOSED account posts and raises REVIEW
(cash was taken)
```

## 8. Screens

Khata page (web, MPC's accounts)  
spec)

Repayment at the till (in M6, from this

```
+-----+
+-----+
| K. Perera . 07xxxxxxx . si . NIC ****1234 . [Consents 2] |
| 6,200.00 |
| Account M042 . OPEN . limit 15,000 [Amend] hard block [ ] |
|
| balance 6,200.00 . available 8,800 . offline cap 5,000 |
|
| Ageing: 0-30 4,200 | 31-60 2,000 | 61-90 0 | 90+ 0 |
|
| Postings: date . kind . amount . receipt/doc . allocated |
+-----+
| 2026-09-01 CHARGE 3,500 RCT M042-S07-T1-0001234 . 3,500 |
| 2026-09-05 PAYMENT -3,500 CPR M042-S07-T1-0001301 . alloc |
| [Record payment] [Adjust...] [Statement] [Suspend] [Close] |
+-----+
```

| Customer: K. Perera balance Rs  
| Amount [ 2,000 ] cash  
| new balance Rs 4,200.00 (provisional)  
| [Record payment] -> receipt prints

| Screen                   | Components                                                                                    | Message ids (m7.) | Notes                                                     |
|--------------------------|-----------------------------------------------------------------------------------------------|-------------------|-----------------------------------------------------------|
| Khata index (lookup)     | search by phone/name, ItemCard-style customer rows; till version from the snapshot            | customer.*        | Reads audited on the web                                  |
| Customer card            | form (minimal fields), consents, phone history with reuse confirmation step, tags, attributes | customer.*        | Registration online only; the till opens this in web mode |
| Khata page               | as sketched; ApprovalBar; StepUpDialog on limit increase; Timeline of postings                | account.*         | —                                                         |
| Charge to account (till) | tender path in M6's sales screen: cap and limit messages from the rules in 7.3                | tender.*          | M6 renders                                                |
| Repayment                | amount, method, reference; provisional balance; offline allowed at the till                   | payment.*         | Reversal on the web with StepUp                           |
| Statement                | period picker, rendered PDF, SMS preview                                                      | statement.*       | —                                                         |
| Bank slip / cash book    | session picker with expected cash, amount, bank, reference, shortfall reason                  | banking.*         | Handover then deposit                                     |

| Screen                 | Components                                                                        | Message ids (m7.) | Notes                    |
|------------------------|-----------------------------------------------------------------------------------|-------------------|--------------------------|
| Privacy requests       | request list, access export download, erasure with guard message and legal ground | privacy.*         | Responsible officer only |
| Receivables and ageing | table by bucket; overdue list; export                                             | receivables.*     | Federation view-only     |

## 9. Tests

| Layer             | Cases                                                                                                                                                                                                                                          | Target          |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Unit              | Every guard of Section 6; ReuseDetector windows and conflicts; Allocator oldest-first and specific incl. partial and over-payment; AgeingCalculator buckets; Anonymiser content; banking expected vs recorded                                  | All branches    |
| Privacy (CI gate) | Event payload scanner: no field matching phone or NIC patterns in any M7 event; snapshot rows contain no nic_* or attributes; logs contain no phone values; access export contains every field the customer's rows hold                        | Zero findings   |
| Property-based    | Balance = $\Sigma$ postings under random sequences incl. reversals; $\Sigma$ allocations per charge $\leq$ charge; unallocated payment amount consistent; ageing sums to open balance                                                          | 1000 cases each |
| Contract          | Every operation; lookup shape is till-safe; RLS: identity readable by an account-holding MPCs, balances OWN only                                                                                                                               | Every operation |
| State machine     | Customer, account, payment, DSAR, banking                                                                                                                                                                                                      | All transitions |
| Consumers         | ACCOUNT tender $\rightarrow$ CHARGE; refund $\rightarrow$ CREDIT; offline breach $\rightarrow$ REVIEW; hard-block bypassed offline $\rightarrow$ ALERT; CPR bundle $\rightarrow$ PAYMENT with allocations; CLOSED account $\rightarrow$ REVIEW | Six cases       |
| Scenario          | Flows 6.1–6.7 of doc 27 incl. the till simulator for offline tender and repayment                                                                                                                                                              | Seven flows     |
| Performance       | Lookup under 50 ms on 500,000 customers; statements for 100,000 accounts under 30 min                                                                                                                                                          | Two targets     |
| Web               | Playwright: register with reuse confirmation $\rightarrow$ open account with step-up $\rightarrow$ record payment $\rightarrow$ statement; access request export                                                                               | Two flows       |

## 10. Tickets in build order

| #     | Ticket                                                                                                            | Est. (days) | Done when                           |
|-------|-------------------------------------------------------------------------------------------------------------------|-------------|-------------------------------------|
| M7-01 | Scaffold; migration with RLS incl. the identity/account visibility split; seeds; privacy CI gate                  | 2           | Gate fails on a planted phone field |
| M7-02 | Customer, phones, consents: register (online-only), change phone with ReuseDetector, deactivate, tags, attributes | 2.5         | Flow 6.1, 6.2                       |
| M7-03 | Accounts: open with NIC capture, limits, hard block, cap, suspend/reinstate/close; LimitPolicy                    | 2           | State machine tests                 |
| M7-04 | PostingService, Allocator, AgeingCalculator; property tests                                                       | 2.5         | Properties green                    |
| M7-05 | ReceiptTenderConsumer; CPR document, RecordCustomerPayment (office), CprBundleHook (till), reversal               | 2.5         | Flow 6.3, 6.4                       |
| M7-06 | Snapshot contributor and change-log fan-out; till contract signed with the till track                             | 1           | Snapshot tests                      |
| M7-07 | Statements (job, builder, notification rule); adjustments with SoD                                                | 1.5         | Flow 6.5                            |
| M7-08 | Privacy requests: access export, erasure with guard and anonymiser, refusal                                       | 1.5         | Flow 6.7; export completeness       |
| M7-09 | Banking records, SessionConsumer, shortfall job; receivables and ageing queries                                   | 1.5         | Flow 6.6                            |
| M7-10 | Web screens (nine) and Playwright flows                                                                           | 3           | Two flows green                     |
| M7-11 | Contract freeze: m7customers.yaml v1; review with M6, M8, M9; CR-27-2 applied to 24A                              | 0.5         | Dependants acknowledge              |

Total about 20 developer-days.

## 11. Open points: what to assume, whom to ask

| Point                                                                                                                                | Assume                           | Decider                               | If different                               |
|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|---------------------------------------|--------------------------------------------|
| Allocation mode, offline repayment capture, limit policy, erasure wording, NIC capture, banking handover (doc 27 DR-1 to DR-5, DR-7) | As proposed in doc 27 §11        | Federation accounts; Federation legal | Config values; Allocator default; H-02 set |
| CPR as an M7 document; payer_kind CUSTOMER removed from 24A (DR-6)                                                                   | Adopted; CR-27-1, CR-27-2 raised | Architecture review                   | –                                          |

| Point                                      | Assume                                                                               | Decider                         | If different                                                                 |
|--------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------|------------------------------------------------------------------------------|
| Identity visibility across MPCS            | Name and phone visible to any MPCS holding an account for the customer; balances OWN | Federation legal (PDPA)         | Restrict to the registering MPCS; second MPCS registers a duplicate identity |
| Retention years for postings after closure | 7 pending J-06                                                                       | Federation legal                | RetentionGuard parameter                                                     |
| Statement SMS content                      | Balance, due amount, oldest charge date, shop name; no line detail                   | Federation with MPCS treasurers | Template                                                                     |

End of document. 27A v0.1; m7customers/README.md and its tests become the living version once merged.